

ESWIN

ESSDK 音频用户手册

Rev 1.0

2026-04-17

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

文档版本	日期	说明
Rev 1.0	2026/04/17	重构文档结构
Rev 0.8	2024/12/20	调整 Audio 软件和硬件章节并增加双 DIE 说明
Rev 0.7.2	2024/12/05	增加硬件章节和软件章节说明
Rev 0.7	2024/08/06	修改章节及部分描述
Rev 0.4	2024/03/27	修改部分错误描述进而缩进问题
Rev 0.2	2024/02/28	更新音频输入、输出，音频编码、解码部分内容
Rev 0.1	2024/01/22	初始版本

目录

声明	1
知识产权声明	1
责任限制	1
修订记录	2
1. 概述	4
1.1. Audio 框架	4
1.2. 核心模块功能说明	4
1.2.1. AI 模块 (Audio Input)	4
1.2.2. AO 模块 (Audio Output)	4
1.2.3. ADEC 模块 (Audio Decoder)	5
1.2.4. AENC 模块 (Audio Encoder)	5
1.3. 重采样 (Resampling)	5
1.4. 声音质量增强 (VQE)	5
1.4.1. AI 模块 VQE 处理链路	5
1.4.2. AO 模块 VQE 处理链路	5
1.4.3. VQE 子模块功能详解	5
1.5. 典型应用场景	6
1.5.1. 录音与播放/编码一体化	6
1.5.2. 解码与播放一体化	7
2. 多媒体框架插件	8
2.1. 支持的音频格式	8
3. 音频硬件特性	9
3.1. 音频输入和输出	9
3.2. 设备节点对应关系	9
3.3. Audio 硬件调试	11
4. Audio 双 DIE 输入输出	12
5. 开发说明	13

1. 概述

本文档主要介绍 EIC77 SoC 相关的音频软件及硬件特性，主要包括 EsSDK 中的 Audio SDK、ffmpeg 插件、GStreamer 插件以及音频硬件特性等内容。

1.1. Audio 框架

音频 SDK 的整体框架如下图所示。

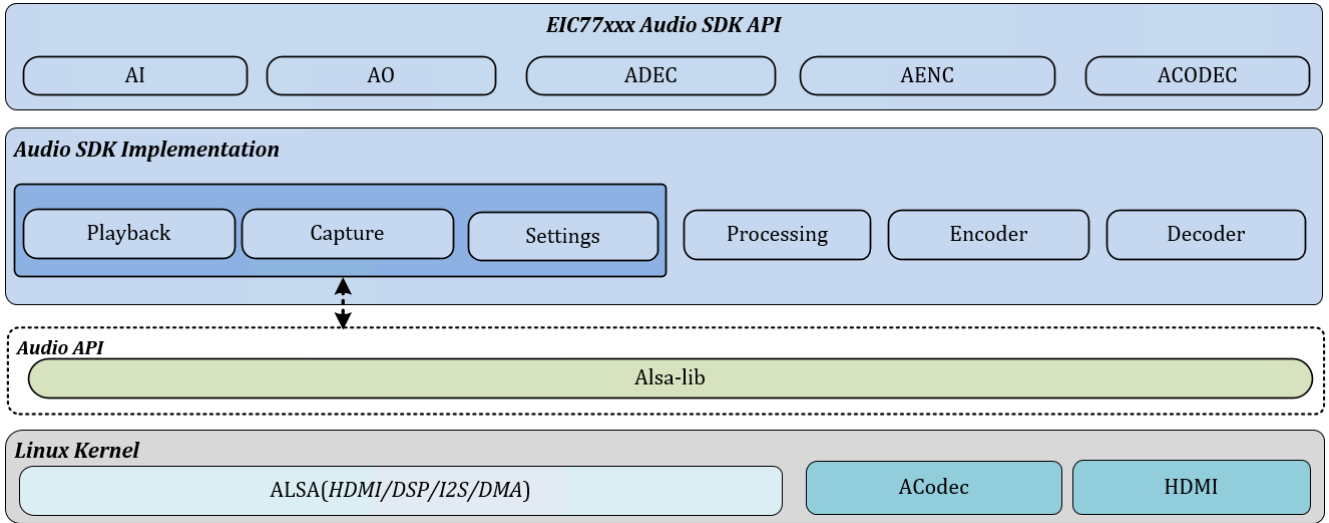


图 1: 音频架构

音频 SDK 通过四个核心模块实现功能：

- AO (Audio Output)：音频播放。
- AI (Audio Input)：音频录制。
- AENC (Audio Encoder)：音频编码。
- ADEC (Audio Decoder)：音频解码。

这些模块均提供同步和异步 API 接口供用户调用，详细用法请参见《ESSDK 音频开发者手册》。

音频 SDK 提供了高效的系统绑定功能，允许模块间数据直接透传，从而大幅提升开发与测试效率。例如，ADEC 模块解码出的 PCM 数据可直接绑定到 AO 模块进行播放，用户仅需调用少量接口即可完成复杂功能。

1.2. 核心模块功能说明

1.2.1. AI 模块 (Audio Input)

AI 模块负责从麦克风等外围设备采集音频数据。

- **硬件支持**：EIC77 SoC 硬件最大支持三路 I2S 设备，可实现最多三路音频输入。
- **配置要点**：使用时需根据实际 Codec 连接情况设置 Card Id 和 Device Id（可通过 `ls /dev/snd` 命令查看）。
- **数据处理**：支持单/双声道，支持 16bit、24bit、32bit 位宽，并集成 VQE 声音质量增强功能。

1.2.2. AO 模块 (Audio Output)

AO 模块负责将 PCM 数据输出到扬声器、HDMI 等设备。

- **硬件支持**：支持三路 I2S 设备输出。
- **配置要点**：需根据 Codec 或 HDMI 的实际连接情况设置 Card Id 和 Device Id。
- **数据处理**：支持单/双声道，支持 16bit、24bit、32bit 位宽，并集成 VQE 声音质量增强功能。

1.2.3. ADEC 模块 (Audio Decoder)

ADEC 模块负责对压缩音频数据进行解码。

- **输出方式**：解码数据既可保存到文件，也可通过**系统绑定**直接输出至 AO 模块播放。
- **支持格式**：G711a、G711 μ 、G722、G726、AAC-LC、AAC-HEV1、AAC-HEV2、AMR、MP3。
- **通道能力**：最多支持创建 16 个解码通道。

1.2.4. AENC 模块 (Audio Encoder)

AENC 模块负责对 PCM 音频数据进行编码。

- **输入方式**：既可编码用户下发的 PCM 数据，也可通过**系统绑定**直接编码 AI 设备录制的音频。
- **支持格式**：G711a、G711 μ 、G722、G726、AAC-LC、AMR。
- **通道能力**：最多支持创建 8 个编码通道。

1.3. 重采样 (Resampling)

AI 和 AO 模块均内置重采样功能，支持任意两种不同采样率（**96kHz 除外**）之间的转换。

- **输入采样率**：8kHz、11.025kHz、12kHz、16kHz、22.05kHz、24kHz、32kHz、44.1kHz、48kHz。
- **输出采样率**：8kHz、11.025kHz、12kHz、16kHz、22.05kHz、24kHz、32kHz、44.1kHz、48kHz。

1.4. 声音质量增强 (VQE)

EIC77 平台支持针对音频输入 (AI) 和音频输出 (AO) 的 **VQE (Voice Quality Enhancement, 声音质量增强)** 功能，旨在显著改善音频质量。VQE 针对 AI 和 AO 的不同应用场景，分别设计了独立的处理链路，并支持对各模块进行独立的开关控制与参数配置。

1.4.1. AI 模块 VQE 处理链路

AI 模块的 VQE 链路主要用于处理麦克风采集的音频信号，其完整流程包含 RES、VOLUME、HPF、EQ、DRC、ANR、AGC 及 AEC 模块，具体流程如下图所示。



图 2: AI 链路

1.4.2. AO 模块 VQE 处理链路

AO 模块的 VQE 链路主要用于优化即将播放的音频信号，其处理流程包含 RES、VOLUME、HPF、EQ、ANR 及 AGC 模块，具体流程如下图所示。



图 3: AO 链路

1.4.3. VQE 子模块功能详解

各 VQE 子模块的具体功能、参数特性及支持场景如下表所示：

模块简称	全称	功能描述	工作采样率	支持声道	开关控制
RES	Resample（重采样）	将输入采样率转换为算法所需的工作采样率（8/16/32/48），处理后再转换为输出采样率。	任意（96kHz 除外）	单/双声道	不支持（固定启用）
VOLUME	Volume（音量调节）	对音频通路进行音量调节，支持静音（Mute）与取消静音（Unmute）功能。	无特定要求	单/双声道	不支持（常开）
HPF	High Pass Filter（高通滤波）	滤除由硬件或工频干扰产生的低频噪声（通常表现为“轰轰”声）。	无特定要求	单/双声道	支持
EQ	Equalizer（均衡器）	对音频数据进行均衡处理，调节各频段的增益。	8/16/32/48kHz	单/双声道	支持
DRC	Dynamic Range Control（动态范围控制）	控制输出电平，将其限制在一定范围内，防止声音忽大忽小。	无特定要求	单/双声道	支持
ANR	Audio Noise Reduction（语音降噪）	在保留语音的同时，去除环境中的背景噪声。	8/16/32/48kHz	单声道	支持
AGC	Auto Gain Control（自动增益控制）	在输入音量波动时，将输出音量稳定在相对一致的范围内。	8/16/32/48kHz	单声道	支持
AEC	Acoustic Echo Cancellation（回声消除）	应用于对讲场景（如 IPC），消除扬声器播放声音在麦克风采集信号中产生的回声。	8/16kHz	单声道	支持

提示

- **AEC 模块**仅存在于 AI 链路中，AO 链路不包含此功能。
- **RES 模块**在 AI 和 AO 链路中均处于强制启用状态，不支持动态关闭。

1.5. 典型应用场景

基于系统绑定机制，音频 SDK 能够构建高效的数据处理流水线，将不同模块串联以实现复杂的业务场景，而无需用户手动搬运数据。

1.5.1. 录音与播放/编码一体化

AI 模块采集音频数据后，经过 VQE 算法处理，可通过系统绑定机制实现下游业务的无缝衔接：

- **录音并播放**：AI 采集的音频经 VQE 处理后，直接绑定至 AO 模块，实时输出至扬声器或 HDMI 设备，实现“边录边播”功能。

- **录音并编码**：AI 采集的音频经 VQE 处理后，直接绑定至 AENC 模块，实时编码生成音频流，供上层 APP 存储或网络传输。

上述 AI 模块通过系统绑定实现 AO 播放或 AENC 编码的数据流向如下图所示。

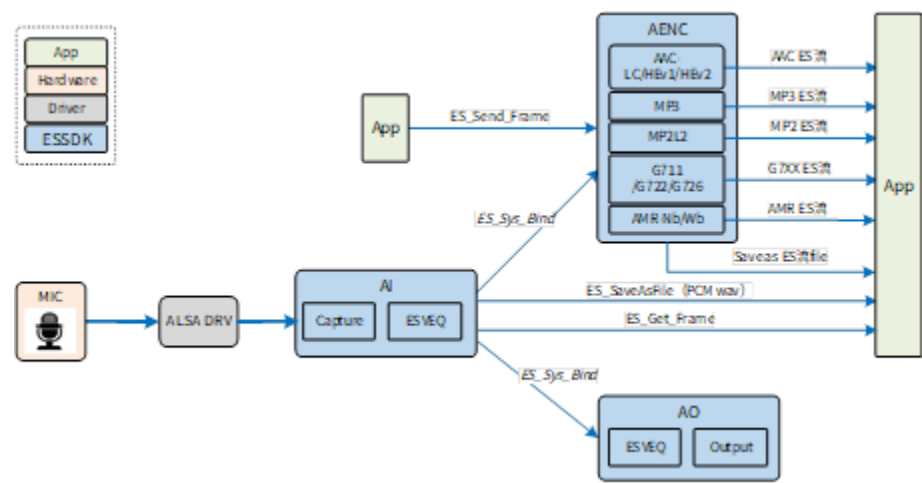


图 4: 场景 1

1.5.2. 解码与播放一体化

除了处理输入源，AO 模块还可接收来自解码模块的数据。通过系统绑定机制，将 ADEC 模块解码后的 PCM 数据直接输送至 AO 模块，从而实现音频流的解码与播放一体化，其处理流程如下图所示。

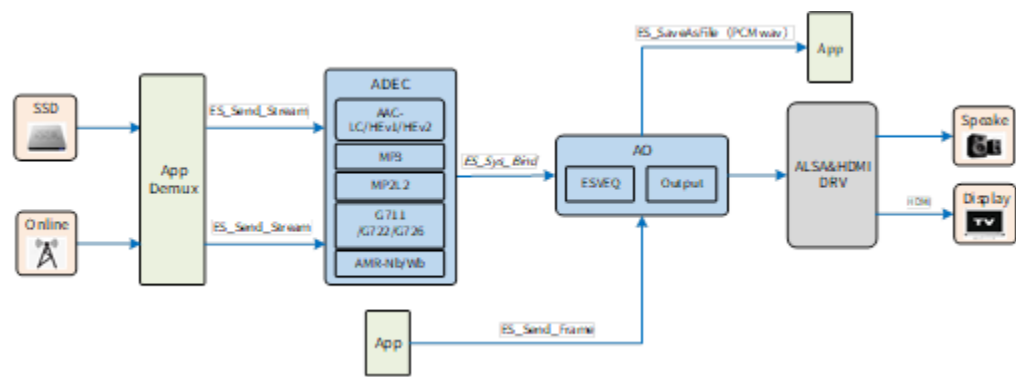


图 5: 场景 2

2. 多媒体框架插件

本节介绍 Essdk 中集成的 **FFmpeg** 和 **GStreamer** 插件。这两个插件均基于各自的开源框架开发，旨在为用户提供便捷、灵活且高效的音频编解码功能，广泛适用于流媒体服务、语音通信和嵌入式音频处理设备 etc 应用场景。

2.1. 支持的音频格式

1. AAC 系列

- **解码**：支持 AAC-LC (Low Complexity) 和 HE-AAC (High Efficiency AAC) 等常见高级音频格式。
- **编码**：目前支持 LC Profile 类型的编码，特别适合对音质有较高要求的场景。

2. G 系列 (G-Series)

- 覆盖 G.711 (A-law/ μ -law)、G.722 及 G.726 等经典语音编码格式，广泛应用于传统语音通信、IP 电话 (VoIP) 等窄带/宽带语音场景。

3. AMR 系列 (Adaptive Multi-Rate)

- 包括 AMR-NB (Narrowband) 和 AMR-WB (Wideband)，专为移动通信设备优化，提供高效的语音压缩方案。

4. MP3 解码

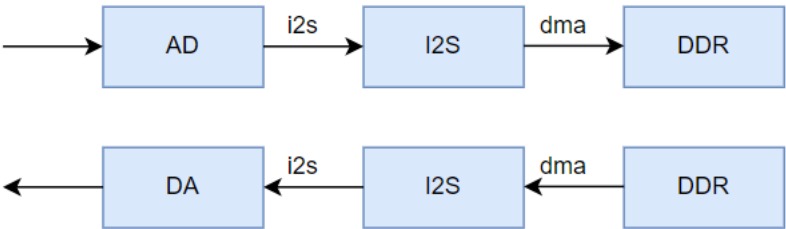
- 支持主流 MP3 格式的解码，适用于音乐播放、在线流媒体内容消费等场景。

3. 音频硬件特性

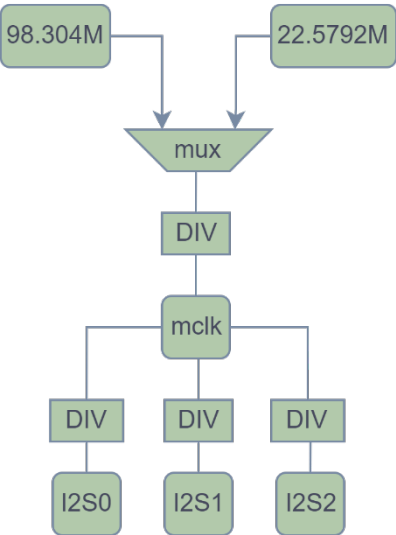
3.1. 音频输入和输出

音频输入的过程为：原始模拟音频信号通过 Codec 按一定的采样率和采样精度转化为数字信号，Codec 以标准 I2S 时序协议，将数字信号传递给 I2S 设备，芯片利用 DMA 将 I2S 设备中的音频数据搬运到内存中，最后完成音频的录制。

音频输出过程与音频输入相似，DMA 将内存中的数据搬运到 I2S 设备中，I2S 设备通过 I2S 协议向 Codec 或 HDMI 送数据，Codec 和 HDMI 再完成后级数据的处理。音频输入输出过程如下图所示。



EIC77 中规定 I2S 为主设备来提供时钟。EIC77 有三路 I2S，共用同一个主时钟，三路 I2S 各有一个独立的分频器，可以分频得到不同的时钟。同一路 I2S 如果工作在全双工模式，那么它们的工作时钟必须相同，硬件时钟如下图所示。

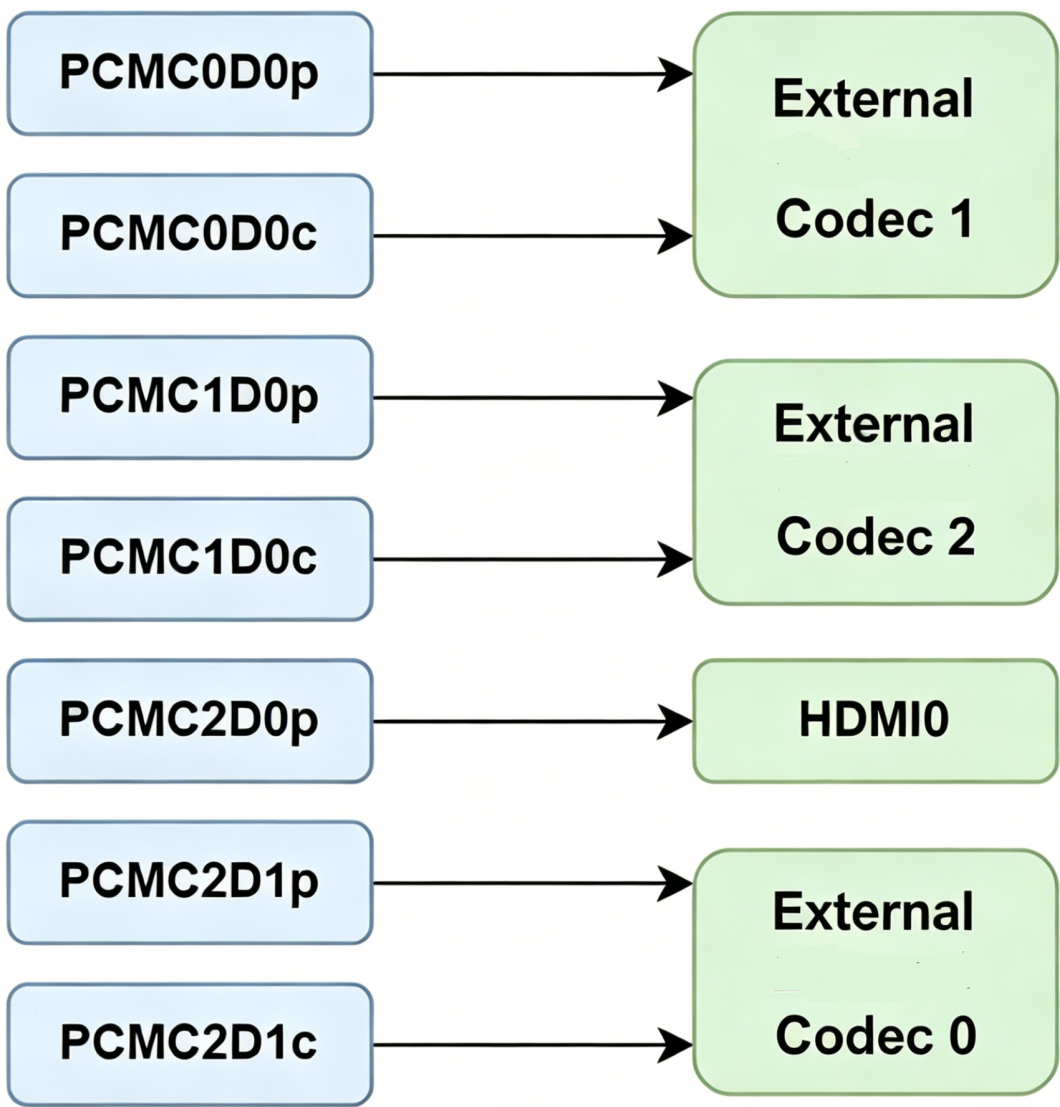


EIC77 支持的三路 I2S 中，每一路 I2S 支持采样率为 8kHz、16kHz、32kHz、44.1kHz、48kHz、96kHz，支持数据位宽为 16bit、24bit、32bit，支持双声道，小端处理。关于 I2S 需要关注以下说明：

- 三路 I2S 共用一个主时钟，44.1kHz 对应的时钟无法和 48kHz 对应的时钟成倍数关系，所以不支持 44.1kHz 和其它采样率（96kHz、48kHz、32kHz、16kHz、8kHz）同时工作。如 I2S0 正在播放 44.1kHz 的音频，I2S1 和 I2S2 也只能播放 44.1kHz 音频；或者当 I2S0 正在播放 48kHz 的音频，I2S1 和 I2S2 不能播放 44.1kHz 音频，只能播放 96kHz、48kHz、32kHz、16kHz、8kHz 音频。
- I2S0 连接 hdmi 的这路 PCM，采样率只支持 96kHz、48kHz、44.1kHz、32kHz，数据位宽支持 16bit、24bit、32bit，支持双声道，小端处理。

3.2. 设备节点对应关系

基于 EIC77 SOC 的不同开发板的硬件外设有所不同，以 EIC770X-EVB-1 开发板为例，其共有 7 个 PCM 节点，其中有 4 个播放节点、3 个录制节点，节点与外设的对应关系如下图所示。



注: EIC7700-EVB-1 开发板的 CODEC0 和 HDMI 使用了同一个 I2S0, 所以同时只能有一个设备工作, 即 PCMC2D0p 和 PCMC2D1p 互斥, 同时只能打开一个节点。如果有三路以上的音频输入和输出需求, 建议使用 USB-hub 实现多路音频的扩展。

基于 EIC77 SOC 的不同开发板音频设备节点与外设的对应关系如下表所示。

音频节点	EVB-1 开发板外设	EVB-2 开发板外设	DVB 开发板外设
PCMC0D0p	CODEC0 Output	CODEC0 Output	CODEC0 Output
PCMC0D0c	CODEC0 Input	CODEC0 Input	CODEC0 Input
PCMC1D0p	CODEC1 Output	CODEC1 Output	HDMI Output
PCMC1D0c	CODEC1 Input	CODEC1 Input	无
PCMC2D0p	HDMI Output	HDMI Output	无
PCMC2D1p	CODEC2 Output	无	无
PCMC2D1c	CODEC2 Input	无	无

3.3. Audio 硬件调试

在音频配置文件 /etc/asound.conf 中，C0D0p 至 C5D0p 是基于 ALSA（Advanced Linux Sound Architecture）系统定义的虚拟 PCM（Pulse Code Modulation）节点。这些虚拟节点通过配置文件中的 plug 类型进行定义，映射到对应的物理音频硬件设备，并利用 dmix 插件实现音频流的混音功能，从而支持多应用程序共享音频设备并高效管理音频资源。每个虚拟节点都映射到一个具体的音频硬件设备，如下表所示，虚拟节点具有以下主要功能：

音频设备抽象

每个节点封装底层硬件设备，简化了上层应用对音频硬件的访问，通过虚拟化，屏蔽了硬件设备差异，为开发者提供统一的接口。

动态音频混合

使用 dmix 插件为每个硬件设备提供混音功能（如多进程共享一个设备），通过虚拟节点，支持多个音频流同时输出到同一物理设备，无需硬件支持多流。

格式转换与适配

通过 plug 插件支持自动格式转换，如采样率、位宽和通道数的调整，确保输入音频流与设备匹配。

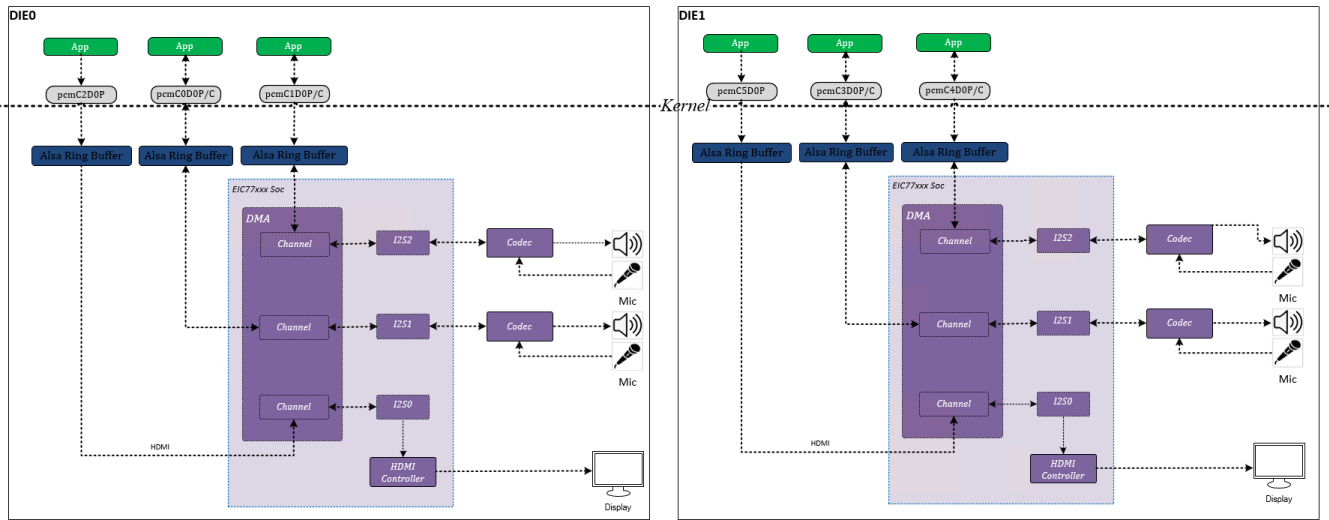
节点名称	物理设备	描述	备注
C0D0p	hw:0,0	对应物理设备 0 的音频设备，处理音频输出流	单/双 die 板子都有效
C1D0p	hw:1,0	对应物理设备 1 的音频设备，处理音频输出流	单/双 die 板子都有效
C2D0p	hw:2,0	对应物理设备 2 的音频设备，处理音频输出流	单/双 die 板子都有效
C3D0p	hw:3,0	对应物理设备 3 的音频设备，处理音频输出流	双 die 板子有效
C4D0p	hw:4,0	对应物理设备 4 的音频设备，处理音频输出流	双 die 板子有效
C5D0p	hw:5,0	对应物理设备 5 的音频设备，处理音频输出流	双 die 板子有效

虚拟节点播放的示例：./aplay -D C3D0p 48k_stereo_32bit.wav

```
eswin@rockos-eswin:~/15_audio_res$ ./aplay -D C3D0p 48k_stereo_32bit.wav
Playing WAVE '48k_stereo_32bit.wav' : Signed 32 bit Little Endian, Rate 48000 Hz, Stereo
```

4. Audio 双 DIE 输入输出

双 DIE 音频的架构图如下所示。



EIC77X 具有双 DIE 芯片版本，双 DIE 芯片在 SoC 架构上集成两套完全相同硬件资源，运行在同一操作系统之上，硬件地址在系统中经过线性化处理，CPU 能够直接高效地访问这些资源。

双 DIE 芯片的可用资源增加一倍，对于音频硬件，I2S 设备增加到 6 个，对应音频节点也对应增加。双 DIE 开发板音频节点与外设的对应关系如下表所示。

音频节点	双 DIE 开发板外设	备注
PCMC0D0p、PCMC0D0c	CODEC0 Output、CODEC0 Input	单/双 die 板子都有效
PCMC1D0p、PCMC1D0c	CODEC1 Output、CODEC1 Input	单/双 die 板子都有效
PCMC2D0p	HDMI0 Output	单/双 die 板子都有效
PCMC3D0p、PCMC3D0c	CODEC2 Output、CODEC2 Input	双 die 板子有效
PCMC4D0p、PCMC4D0c	CODEC3 Output、CODEC3 Input	双 die 板子有效
PCMC5D0p	HDMI1 Output	双 die 板子有效

在双 DIE 环境下，由于跨 DIE 数据访问的性能会影响性能，因此在系统运行时，应尽量减少跨 DIE 的数据传输，以提升整体运行效率。因此在软件上，设置了 CARD0 到 CARD2 应用到 DIE 0 做处理，CARD3 到 CARD5 应用到 DIE 1 做处理。

5. 开发说明

EIC77 提供的音频 SDK 包括开发所必要的头文件和库，另外 SDK 提供了音频开发的参考示例 sample_audio，基于 sample_audio 可以进行 AI、AO、ADEC、AENC 模块 SDK 的开发，音频 SDK 开发包相关文件如下所示。

```

├── bin
│   └── sample_audio
├── include
│   └── audio
│       ├── es_audio.h
│       ├── es_comm_adec.h
│       ├── es_comm_aenc.h
│       ├── es_comm_aio.h
│       └── es_debug.h
├── lib
│   └── audio
│       ├── libes_audio_control.so
│       ├── libes_audio_hal.so
│       ├── libes_audio_processing.so
│       ├── libes_audio_sdk.so
│       ├── libes_audio.so
│       ├── libes_codec_processor.so
│       ├── libes_codec.so
│       ├── libes_params_convert.so
│       ├── libes_vqe.so
│       ├── libes_aec.so
│       ├── libes_agc.so
│       ├── libes_hpf.so
│       ├── libes_drc.so
│       ├── libes_eq_iir.so
│       ├── libes_mux.so
│       ├── libes_ns.so
│       ├── libes_src.so
│       └── libes_volume.so
├── samples
│   └── audio_sample
│       ├── adp
│       │   ├── audio_adpcm_adp.c
│       │   └── audio_adpcm_adp.h
│       ├── utils
│       │   ├── utils.c
│       │   └── utils.h
│       ├── build.sh
│       ├── CMakeLists.txt
│       ├── README.md
│       ├── data
│       │   └── audio
│       │       ├── 48000_16bit_setreo_4000_10000ms.wav
│       │       ├── 48000_32bit_setreo_4000_10000ms.wav
│       │       └── music.wav
│       ├── sample_audio.c
│       ├── sample_comm_audio.c
│       └── sample_comm.h

```

对 SDK 开发包目录有如下说明：

- **bin**: sample_audio 为参考示例的可执行文件，可以在开发板上进行测试，用来验证 AI、AO、AENC 及 ADEC 的功能。
- **include/audio**: 包含 SDK 开发所需要依赖的头文件，在编写代码时需要包含头文件，其中 es_audio.h 为 AI、AO、AENC、ADEC 模块的接口头文件。
- **lib/audio**: 包含所需要依赖的库，包括 VQE 相关的算法库，在运行时需要依赖算法库。

- **samples/audio_sample**: 参考示例 sample_audio 所对应的源文件及测试文件，基于所提供的源码进行编译，可以生成可执行文件 sample_audio，在开发板上进行相关 AUDIO 功能的测试。

关于 sample_audio 对应如下开发示例：

- AI 模块相关的示例参考 SAMPLE_AUDIO_AI，该示例介绍了音频录制的整体流程以及使用方式。
- AO 模块开发相关示例参考 SAMPLE_AUDIO_AO，该示例介绍了音频播放的整体流程以及使用方式。
- AENC 模块开发相关示例参考 SAMPLE_AUDIO_AENC，该示例介绍了音频编码的整体流程以及使用方式。
- ADEC 模块开发相关示例参考 SAMPLE_AUDIO_ADEC，该示例介绍了音频解码的整体流程以及使用方式。
- 系统绑定相关的示例参考 SAMPLE_AUDIO_AiAo, SAMPLE_AUDIO_AiAenc, SAMPLE_AUDIO_AdecAo, 这些示例介绍了系统绑定的调用流程以及使用方式。